SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) FINAL REPORT

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR CINEMA SCREENING AND TICKET BOOKING SYSTEM

Academic Year 2026–2027

Submitted by

G. Sahasra Reddy

Roll Number: 25WU0102080

Section: AIML Rhinos

Date of Submission: 15 October 2026

Woxsen University

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing me with the opportunity to undertake this Project-Based Learning activity as part of the Database Management Systems course.

I thank the course faculty for their guidance and suggestions throughout the analysis, design and documentation of the Cinema Screening and Ticket Booking System. The review process helped me connect database concepts such as ER modelling, relational schema design, normalization, SQL constraints, transactions and application connectivity with a practical booking scenario.

I also acknowledge the learning resources and software documentation consulted while preparing the design and SQL examples. This project has strengthened my understanding of how a structured relational database can improve data consistency, seat allocation and operational reporting.

TABLE OF CONTENTS

Abstract

List of Tables

List of Figures

List of Acronyms

CHAPTER 1 — PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

1.2 Objectives

1.3 Scope and Assumptions

1.4 Proposed Solution Strategy

1.5 Functional Requirements

1.6 Entity-Relationship Diagram (ERD)

1.7 Entity and Relationship Summary

CHAPTER 2 — LOGICAL DATABASE DESIGN

2.1 Relational Schema

2.2 Keys and Integrity Constraints

2.3 Functional Dependencies

2.4 Normalization up to 3NF

2.5 Data Dictionary

2.6 Implementation of Logical Design: DDL and Sample Data

CHAPTER 3 — APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

3.2 Development Environment and Database Connectivity

3.3 Application Modules and Database Mapping

3.4 CRUD Operations

3.5 Business Rules, Validation and Error Handling

3.6 Reports and Database Operations

3.7 Application Screens

3.8 Testing and Results

CHAPTER 4 — CONCLUSION

4.1 Limitations and Future Enhancements

5. References

6. Appendices

Note: Update page numbers automatically in Microsoft Word after final pagination.

ABSTRACT

Cinema operations require accurate coordination of movies, screens, screenings, seats, customer bookings, tickets, concessions and payments. Manual registers or disconnected spreadsheets can cause duplicate seat allocation, scheduling conflicts, inconsistent payment records and slow report generation. This project proposes a centralized relational database management system for cinema screening and ticket booking. The design identifies the main entities and their relationships, maps the conceptual model to relational tables, and applies normalization principles up to Third Normal Form (3NF). MySQL is selected as the target DBMS and Python is proposed for the application layer. Primary and foreign keys, uniqueness rules, checks, transactions and parameterized queries are used to protect data integrity. The planned application modules cover cinema and screen setup, movie and show scheduling, seat availability, booking, ticket issue, concession orders, payment records, cancellations, refunds and operational reports. Representative SQL demonstrates joins, aggregation, nested queries, views and indexes. Review material reports passing checks for duplicate seat booking, invalid concession quantity, overlapping screenings, invalid booking identifiers and excessive refunds; transaction rollback remains not tested, and payment gateway integration is not tested. The project demonstrates a structured approach to more reliable booking management and provides a foundation for future application completion.

Keywords: Cinema Booking, Relational Database, ER Diagram, 3NF Normalization, MySQL, Python

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Cinema chains maintain records for cinemas, screens, seats, movies, screening schedules, customers, bookings, tickets, concession sales, payments, cancellations and refunds. When these records are maintained in spreadsheets, manual registers or separate systems, staff may have difficulty identifying current seat availability, preventing schedule conflicts, matching payments to bookings and preparing reliable reports. Manual seat allocation can also allow the same seat to be assigned to more than one customer for the same screening.

The project addresses the need for a centralized relational database that stores cinema operations in related tables and applies integrity rules. The proposed system is intended for cinema administrators, customers and box-office staff. It aims to reduce duplicate records, improve retrieval, support consistent booking workflows and provide useful operational summaries.

1.2 Objectives

- Design an ER model and relational schema for cinema, screening, booking and post-booking data.
- Normalize the database design up to Third Normal Form and define suitable keys and constraints.
- Prepare SQL statements for table creation, sample data, joins, nested queries, aggregation, views and indexes.
- Design a Python application connected to MySQL for scheduling, seat selection, bookings, concessions, payments, cancellations and refunds.
- Validate important business rules and prepare reports for occupancy, movie sales, screen utilization, concessions, cancellations and revenue.

1.3 Scope and Assumptions

Item	Description
In scope	Cinema and screen records; seat layout; movie and screening schedule; customer details; bookings and booked seats; tickets; concession items and orders; payments; cancellations and refunds; search and reports.
Out of scope / exclusions	Live payment-gateway processing, production deployment, external ticket-provider integration, barcode-scanner hardware and multi-currency settlement are not demonstrated in the review evidence.
Assumptions	Each screening is assigned to one screen and one movie. Each seat belongs to one screen. A seat can be booked only once for a given screening. Booking and payment state changes must be recorded consistently. Refunds cannot exceed eligible paid amounts.
Implementation status	Review 3 describes Python + MySQL application design. Several validation tests are reported as PASS; transaction rollback is marked NOT TESTED and payment-gateway integration is NOT TESTED. Illustrative UI mockups are not treated as evidence of a fully deployed application.

1.4 Proposed Solution Strategy

The proposed solution separates data storage from application logic. MySQL stores normalized tables and enforces relational constraints. A Python application provides user-facing forms and workflows, validates inputs, executes parameterized SQL, and commits or rolls back transactions. Frequently searched fields such as screening identifiers, booking identifiers and foreign-key columns can be indexed after reviewing query patterns.

Solution Element	Project-Specific Description
Primary users	Cinema administrator, customer and box-office staff.
Major modules	Cinema/screens, movies/shows, seats/bookings, tickets/ concessions, payments/cancellations/refunds, reporting.
Workflow	Configure cinema and screen → add movie and screening → check seat availability → select seats → validate and create booking → record payment → issue tickets → optionally order concessions → process eligible cancellation/refund.
Business rules	No duplicate seat for a screening; no overlapping shows on one screen; positive concession quantity; valid cancellation deadline; refund within eligible payment; start time precedes end time.
Expected reports	Occupancy by screening, movie-wise ticket sales, screen utilization, concession revenue by item/date, cancellation and refund summaries.

1.5 Functional Requirements

Req. ID	Functional Requirement	Priority
FR-01	Administrator can create, view, update and remove cinema and screen records.	High
FR-02	Administrator can maintain movies and screening schedules.	High
FR-03	System stores seat layouts and screen capacity.	High
FR-04	System checks seat availability for a selected screening.	High
FR-05	Customer or staff can book available seats.	High
FR-06	System records booked seats and issues ticket records.	High
FR-07	System records concession items, quantities and order amounts.	Medium
FR-08	System records booking and concession payment details.	High
FR-09	System accepts cancellations only when policy conditions are satisfied.	High
FR-10	System calculates and records eligible refunds.	High
FR-11	Users can search movies, screenings and bookings.	Medium
FR-12	Authorized users can generate operational and revenue reports.	Medium

1.6 Entity-Relationship Diagram (ERD)

Figure 1 summarizes the main entities and their relationships. The schema uses separate entities for each major subject and junction records for seat allocation and concession order lines. The final implementation should keep the ERD, table names and keys synchronized.

Relationship	Cardinality	Meaning
Cinema — Screen	1 : many	One cinema contains multiple screens; each screen belongs to one cinema.
Screen — Seat	1 : many	One screen contains multiple seats; each seat belongs to one screen.
Movie — Screening	1 : many	A movie may have many screenings; each screening presents one movie.
Screen — Screening	1 : many over time	A screen hosts multiple screenings at different times.

Customer — Booking	1 : many	A customer may create multiple bookings.
Screening — Booking	1 : many	A screening may have multiple bookings.
Booking — Booked_Seat	1 : many	A booking may reserve multiple seats.
Booked_Seat — Ticket	1 : 1 in the proposed design	Each booked-seat record can have one ticket record.
Booking — Concession_Order	1 : many	A booking may have multiple concession order lines.
Booking — Payment	1 : many	A booking can have payment records for its transactions.
Booking — Cancellation	0 : 1 intended	A booking may have no cancellation or one cancellation record.
Cancellation — Refund	0 : 1 intended	An eligible cancellation may result in a refund record.

Figure 1. Entity-Relationship Diagram of the Proposed System — relationship summary

ERD note: A graphical ER diagram should be inserted here before submission if the faculty requires a visual diagram. The relationships above reflect the entity/key structure documented in the review slides.

1.7 Entity and Relationship Summary

Entity	Major Attributes / Key	Purpose / Rule
CINEMA	Cinema_ID (PK), Name, Location, Total_Screens	Stores cinema branch information.
SCREEN	Screen_ID (PK), Cinema_ID (FK), Screen_Name, Screen_Type, Total_Seats	Defines a screen within a cinema.
SEAT	Seat_ID (PK), Screen_ID (FK), Seat_Number, Seat_Type	Stores seats; Screen_ID + Seat_Number should be unique.
MOVIE	Movie_ID (PK), Title, Genre, Duration, Language	Stores movie details.
SCREENING	Screening_ID (PK), Movie_ID (FK), Screen_ID (FK), Show_Date, Start_Time, End_Time	Schedules a movie on a screen; time conflicts must be rejected.
CUSTOMER	Customer_ID (PK), Name, Email, Phone	Stores customer contact information.
BOOKING	Booking_ID (PK), Customer_ID (FK), Screening_ID (FK), Booking_Date, Status	Represents a booking for a screening.
BOOKED_SEAT	Booked_Seat_ID (PK), Booking_ID (FK), Seat_ID (FK)	Associates a booking with a seat; unique screening-seat allocation is enforced in application logic/transaction.
TICKET	Ticket_ID (PK), Booked_Seat_ID (FK), Price, Issue_Date	Stores ticket issue details.
CONCESSION_ITEM	Item_ID (PK), Item_Name, Price	Stores snack and beverage products.
CONCESSION_ORDER	Order_ID (PK), Booking_ID (FK), Item_ID (FK), Quantity, Amount	Stores concession order lines.
PAYMENT	Payment_ID (PK), Booking_ID (FK), Amount, Payment_Date, Payment_Method, Status	Records payment transactions.
CANCELLATION	Cancellation_ID (PK), Booking_ID (FK), Cancellation_Date, Reason	Records cancellation requests.
REFUND	Refund_ID (PK), Cancellation_ID (FK), Refund_Amount, Refund_Date	Records eligible refunds.

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

The following schema is the logical design proposed for the system. PK denotes primary key and FK denotes foreign key. Composite uniqueness is used where a natural business rule requires it.

S.No.	Table Name	Attributes / Keys	Purpose
1	CINEMA	Cinema_ID PK, Name, Location, Total_Screens	Cinema branch details
2	SCREEN	Screen_ID PK, Cinema_ID FK, Screen_Name, Screen_Type, Total_Seats	Screens within a cinema
3	SEAT	Seat_ID PK, Screen_ID FK, Seat_Number, Seat_Type	Seat inventory
4	MOVIE	Movie_ID PK, Title, Genre, Duration, Language	Movie catalogue
5	SCREENING	Screening_ID PK, Movie_ID FK, Screen_ID FK, Show_Date, Start_Time, End_Time	Scheduled screenings
6	CUSTOMER	Customer_ID PK, Name, Email UNIQUE, Phone	Customer records
7	BOOKING	Booking_ID PK, Customer_ID FK, Screening_ID FK, Booking_Date, Status	Booking header
8	BOOKED_SEAT	Booked_Seat_ID PK, Booking_ID FK, Seat_ID FK, Price	Seat allocation per booking
9	TICKET	Ticket_ID PK, Booked_Seat_ID FK UNIQUE, Issue_Date, Ticket_Code UNIQUE	Ticket per booked seat
10	CONCESSION_ITEM	Item_ID PK, Item_Name UNIQUE, Price	Concession catalogue
11	CONCESSION_ORDER	Order_ID PK, Booking_ID FK, Item_ID FK, Quantity, Unit_Price, Amount	Concession order line
12	PAYMENT	Payment_ID PK, Booking_ID FK, Amount, Payment_Date, Payment_Method, Status	Payment ledger
13	CANCELLATION	Cancellation_ID PK, Booking_ID FK UNIQUE, Cancellation_Date, Reason, Status	Cancellation record
14	REFUND	Refund_ID PK, Cancellation_ID FK UNIQUE, Refund_Amount, Refund_Date, Status	Refund record

2.2 Keys and Integrity Constraints

Constraint	Application in this design	Business rule enforced
PRIMARY KEY	Unique identifier in each table, e.g., Cinema_ID, Booking_ID.	Every row can be identified uniquely.
FOREIGN KEY	Screen.Cinema_ID → Cinema.Cinema_ID; Booking.Customer_ID → Customer.Customer_ID; Booking.Screening_ID → Screening.Screening_ID.	Prevents references to missing parent records.
NOT NULL	Required names, titles, dates, identifiers, quantity and monetary values.	Required information cannot be omitted.
UNIQUE	Customer.Email; (Screen_ID, Seat_Number); Ticket.Booked_Seat_ID; Ticket_Code.	Prevents duplicate email/seat labels and duplicate ticket mapping.
CHECK	Duration > 0; Total_Seats > 0; Quantity > 0; Price >= 0; Amount >= 0; End_Time > Start_Time.	Rejects invalid numeric and time values.
DEFAULT	Booking status defaults to PENDING; payment status defaults to PENDING; booking date defaults to current timestamp.	Provides consistent initial values.
Transactions / triggers	Check screening-seat availability and screen-time overlap before committing.	Protects against concurrent double booking and schedule conflict.

2.3 Functional Dependencies

Functional dependencies identify attributes determined by a key. They guide decomposition and help prevent repeated facts from being stored in multiple places.

- Cinema_ID → Name, Location, Total_Screens
- Screen_ID → Cinema_ID, Screen_Name, Screen_Type, Total_Seats
- (Screen_ID, Seat_Number) → Seat_ID, Seat_Type
- Movie_ID → Title, Genre, Duration, Language
- Screening_ID → Movie_ID, Screen_ID, Show_Date, Start_Time, End_Time
- Customer_ID → Name, Email, Phone
- Booking_ID → Customer_ID, Screening_ID, Booking_Date, Status
- Booked_Seat_ID → Booking_ID, Seat_ID, Price
- Item_ID → Item_Name, Price

- Payment_ID → Booking_ID, Amount, Payment_Date, Payment_Method, Status
- Cancellation_ID → Booking_ID, Cancellation_Date, Reason, Status
- Refund_ID → Cancellation_ID, Refund_Amount, Refund_Date, Status

2.4 Normalization up to Third Normal Form (3NF)

The schema is designed with normalization principles in mind. Final compliance should be verified against the actual implemented SQL schema and sample data.

Normal Form	Requirement	Application to the project
First Normal Form (1NF)	Each attribute contains one atomic value; no repeating groups.	Seat allocations and concession items are stored as separate rows rather than comma-separated lists in a booking.
Second Normal Form (2NF)	In 1NF, and each non-key attribute depends on the whole candidate key.	Core entities use single-column surrogate primary keys; order lines and seat allocations are represented as individual records.
Third Normal Form (3NF)	In 2NF, with no transitive dependency of non-key attributes on a key.	Movie, cinema, screen, customer and concession details are stored in their own tables and referenced by foreign keys instead of copied into bookings.
BCNF note	Every determinant should be a candidate key.	The design aims to meet this for the listed functional dependencies; a formal proof depends on the final deployed schema.

2.5 Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
CINEMA	Cinema_ID	INT	PK	Unique cinema identifier
CINEMA	Name	VARCHAR(100)	NOT NULL	Cinema name
CINEMA	Location	VARCHAR(150)	NOT NULL	Cinema address or locality
CINEMA	Total_Screens	INT	CHECK > 0	Number of screens
SCREEN	Screen_ID	INT	PK	Unique screen identifier
SCREEN	Cinema_ID	INT	FK, NOT NULL	Owning cinema
SCREEN	Screen_Name	VARCHAR(50)	NOT NULL	Screen label
SCREEN	Total_Seats	INT	CHECK > 0	Declared seat capacity
SEAT	Seat_ID	INT	PK	Unique seat identifier
SEAT	Screen_ID	INT	FK, NOT NULL	Screen containing seat
SEAT	Seat_Number	VARCHAR(10)	NOT NULL	Seat label
SEAT	Seat_Type	VARCHAR(30)	DEFAULT Standard	Seat category
MOVIE	Movie_ID	INT	PK	Unique movie identifier
MOVIE	Title	VARCHAR(150)	NOT NULL	Movie title
MOVIE	Genre	VARCHAR(50)	NOT NULL	Movie genre
MOVIE	Duration	INT	CHECK > 0	Duration in minutes
MOVIE	Language	VARCHAR(50)	NOT NULL	Movie language
SCREENING	Screening_ID	INT	PK	Unique screening identifier
SCREENING	Movie_ID	INT	FK, NOT NULL	Movie being screened
SCREENING	Screen_ID	INT	FK, NOT NULL	Screen used
SCREENING	Show_Date	DATE	NOT NULL	Screening date
SCREENING	Start_Time / End_Time	TIME	CHECK End_Time > Start_Time	Scheduled time range
CUSTOMER	Customer_ID	INT	PK	Unique customer identifier
CUSTOMER	Name	VARCHAR(100)	NOT NULL	Customer name
CUSTOMER	Email	VARCHAR(150)	UNIQUE	Customer email address
CUSTOMER	Phone	VARCHAR(20)	NULL allowed	Contact number
BOOKING	Booking_ID	INT	PK	Unique booking identifier
BOOKING	Customer_ID / Screening_ID	INT	FK, NOT NULL	Customer and screening references
BOOKING	Booking_Date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Booking timestamp
BOOKING	Status	VARCHAR(20)	DEFAULT PENDING	Booking state
BOOKED_SEAT	Booked_Seat_ID	INT	PK	Unique allocation record
BOOKED_SEAT	Booking_ID / Seat_ID	INT	FK, NOT NULL	Booking and seat references
BOOKED_SEAT	Price	DECIMAL(10,2)	CHECK >= 0	Price charged for the seat
TICKET	Ticket_ID	INT	PK	Unique ticket identifier
TICKET	Booked_Seat_ID	INT	FK, UNIQUE	One ticket per booked seat

TICKET	Ticket_Code	VARCHAR(40)	UNIQUE	Ticket reference code
CONCESSION_ITEM	Item_ID	INT	PK	Unique item identifier
CONCESSION_ITEM	Item_Name	VARCHAR(100)	UNIQUE	Snack or beverage name
CONCESSION_ITEM	Price	DECIMAL(10,2)	CHECK >= 0	Current item price
CONCESSION_ORDER	Order_ID	INT	PK	Unique order-line identifier
CONCESSION_ORDER	Quantity	INT	CHECK > 0	Number of units
CONCESSION_ORDER	Amount	DECIMAL(10,2)	CHECK >= 0	Line total
PAYMENT	Payment_ID	INT	PK	Unique payment record
PAYMENT	Booking_ID	INT	FK, NOT NULL	Booking being paid
PAYMENT	Amount	DECIMAL(10,2)	CHECK >= 0	Amount recorded
CANCELLATION	Cancellation_ID	INT	PK	Unique cancellation record
CANCELLATION	Booking_ID	INT	FK, UNIQUE	Booking being cancelled
REFUND	Refund_ID	INT	PK	Unique refund record
REFUND	Refund_Amount	DECIMAL(10,2)	CHECK >= 0	Refund value

2.6 Implementation of Logical Design: DDL and Sample Data

Item	**Details**
DBMS	MySQL — target database specified in Review 3
Programming language	Python — application design specified in Review 3
Connector	mysql.connector, using parameterized SQL
Tables	14 proposed core tables
Sample data	Representative cinema, screen, seat, movie, screening, customer, booking, concession and payment rows are included in Appendix A.

The complete representative schema and sample SQL are included in Appendix A. Before submission, execute the script in the selected MySQL version and capture the actual successful execution output. The script is a documented baseline and may require adaptation to the exact implementation environment.

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The proposed application uses Python as the presentation and application layer and MySQL as the persistent data layer. It is intended to let administrators maintain cinema data and schedules, while customers or box-office staff check availability, create bookings, issue tickets, add concession items, record payments and handle eligible cancellations or refunds. The application should retrieve and update the normalized tables rather than rely on static sample values.

3.2 Development Environment and Database Connectivity

Item	Details
Programming language	Python
DBMS	MySQL
IDE / GUI toolkit	Not specified in the supplied review material; enter the actual tool used.
Connector / driver	mysql.connector (shown in Review 3)
Operating environment	Not specified in the supplied review material.
Repository link	Not provided in the supplied review material; add the actual GitHub URL if available.

The application should read database credentials from environment variables or a protected configuration file, use parameterized SQL to reduce SQL-injection risk, close cursors and connections safely, commit successful changes, and roll back a transaction when an exception occurs. A connection example is provided in Appendix A.

3.3 Application Modules and Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Cinema and screens	Maintain venues, screens and seat capacity.	CINEMA, SCREEN, SEAT	Create, read, update, controlled delete
Movies and shows	Maintain movie catalogue and schedules.	MOVIE, SCREENING, SCREEN	CRUD; overlap validation
Customers and bookings	Search customers and create bookings.	CUSTOMER, BOOKING	Create and search
Seat availability	Display free seats for a screening.	SCREENING, SEAT, BOOKED_SEAT, BOOKING	Read and transactional allocation
Tickets	Issue ticket references for confirmed seats.	TICKET, BOOKED_SEAT	Create and read
Concessions	Maintain items and add order lines.	CONCESSION_ITEM, CONCESSION_ORDER	CRUD and totals
Payments	Record payment attempts and status.	PAYMENT, BOOKING	Create and read
Cancellations and refunds	Validate policy and record eligible refunds.	CANCELLATION, REFUND, BOOKING, PAYMENT	Validate and transactional update
Reports	Generate operational summaries.	Multiple tables	JOIN, GROUP BY, aggregate, views

3.4 CRUD Operations

Operation	Application Feature / Screen	Tables Affected	Evidence / Notes
Create	Add cinema, movie, customer, screening, booking or order.	Relevant entity tables	CRUD functionality is an objective; retain screenshots of actual execution.
Read / Search	Search movies, screenings, seats, bookings and reports.	MOVIE, SCREENING, SEAT, BOOKING and related tables	Review slides specify search/ report features.
Update	Update schedules, contact details, item prices and statuses.	SCREEN, MOVIE, SCREENING, CUSTOMER, CONCESSION_ITEM, BOOKING	Validate changes and preserve referential integrity.
Delete / Controlled removal	Remove only eligible records; use status changes for historical bookings.	Master tables and transaction tables	Avoid deleting records referenced by bookings or financial records.

3.5 Business Rules, Validation and Error Handling

Rule / Validation	How It Should Be Implemented	Review Evidence / Status
No duplicate seat per screening	Before booking, check the screening-seat combination and protect allocation with a unique design or transaction-safe validation.	Duplicate seat booking: PASS reported.
No overlapping screenings	Compare proposed time interval with existing screenings on the same screen.	Overlapping screenings: PASS reported.
Positive concession quantity	Reject quantity <= 0 with application validation and CHECK constraint.	Zero concession quantity: PASS reported.
Cancellation deadline	Check show time and configured cancellation policy before modifying records.	Policy is specified; no separate test status supplied.
Refund limit	Calculate eligible refund from payment and policy; reject any refund exceeding	Refund above eligibility: PASS reported.
Invalid booking ID	Validate existence and display a clear error without changing data.	Invalid booking ID: error message PASS reported.
Atomic booking transaction	Create booking, allocate seats and record related changes in one transaction; rollback on failure.	Failed booking transaction rollback: NOT TESTED.
Payment gateway	Keep gateway integration separate from local payment record logic.	Payment gateway: NOT TESTED.

3.6 Reports and Database Operations

Report / Operation	SQL / Database Concept	Application Output
Occupancy by screening	COUNT booked seats / screen capacity; GROUP BY screening.	Seats sold and occupancy percentage.
Movie-wise ticket sales	JOIN MOVIE, SCREENING, BOOKING, BOOKED_SEAT and TICKET; SUM ticket price.	Ticket sales and booking count per movie.
Screen utilization	Compare scheduled screen time with available operating time.	Utilization by screen and date.
Concession revenue	JOIN CONCESSION_ORDER and CONCESSION_ITEM; SUM line amount.	Sales by item and date.
Cancellations and refunds	JOIN CANCELLATION, REFUND and BOOKING.	Cancellation count and refund totals.
Customers with multiple bookings	Nested query or GROUP BY / HAVING.	Customer list meeting the booking threshold.
Reusable summaries	CREATE VIEW for movie sales or occupancy.	Consistent reusable report queries.
Query performance	Indexes on frequently searched foreign keys and date/screen fields.	Faster filtering and joins when supported by execution plans.

3.7 Application Screens

The Review 3 presentation contains illustrative mockups for show scheduling, seat availability and concessions/payments. These should be labelled as illustrative rather than screenshots of a verified deployed interface unless actual application captures are available.

Figure 2. Application architecture: Python interface → application validation and booking logic → MySQL tables/queries/transactions → results returned to the user.

Figure 3. Booking workflow: select movie and screening → choose seats → validate availability → commit booking and seat allocation → record payment → issue tickets.

Figure 4. Cancellation workflow: locate booking → verify status and policy → calculate eligible refund → commit cancellation/refund → release seats according to policy.

Figure 5. Application screen categories: cinema/screen management, movie scheduling, seat selection, booking/ticket, concessions/payment and reports.

3.8 Testing and Results

The following table records the statuses explicitly shown in the Review 3 material. “PASS” is reported by the review presentation; “NOT TESTED” is retained where the source states that a test was not performed. Other rows are recommended additional tests and must be updated with actual results before final submission.

Test ID	Feature	Input / Action	Expected Result	Actual Result / Status
TC-01	Duplicate seat booking	Attempt to book a seat already allocated for the same screening.	Reject second allocation.	PASS reported in Review 3.
TC-02	Concession quantity	Enter quantity 0.	Reject invalid quantity.	PASS reported in Review 3.
TC-03	Screening overlap	Schedule overlapping shows on one screen.	Reject conflicting schedule.	PASS reported in Review 3.
TC-04	Invalid booking ID	Search with a non-existent booking identifier.	Show error; do not update records.	Error message — PASS reported in Review 3.
TC-05	Refund limit	Request refund above eligible amount.	Reject refund.	PASS reported in Review 3.
TC-06	Failed booking transaction	Force a failure after booking starts but before commit.	Rollback all partial changes.	NOT TESTED in Review 3.
TC-07	Payment gateway	Attempt external gateway payment.	Return verified gateway result.	NOT TESTED in Review 3.
TC-08	Valid booking	Book available seats for a valid screening.	Booking, seat allocation and tickets recorded consistently.	Execute and record actual result before submission.

TC-09	Cancellation deadline	Attempt cancellation after the allowed deadline.	Reject cancellation without changing records.	Execute and record actual result before submission.

Reported test outcomes indicate that several key validation checks have been considered. However, the final implementation should include evidence for successful end-to-end booking, payment recording, cancellation, refund and transaction rollback before these workflows are described as fully verified.

CHAPTER 4

CONCLUSION

The Cinema Screening and Ticket Booking System applies relational database concepts to a practical cinema-management problem. The project identifies the main operational entities, maps them to a relational schema, records functional dependencies and describes normalization up to 3NF. The design uses primary and foreign keys, uniqueness rules, checks, defaults and transaction-aware application logic to improve consistency across screenings, bookings, seat allocations, tickets, concessions, payments, cancellations and refunds.

The Review 3 material outlines a Python–MySQL application architecture and reports passing tests for duplicate seat booking, invalid concession quantity, overlapping screenings, invalid booking identifiers and refunds above eligibility. It also explicitly identifies transaction rollback and payment-gateway integration as not tested. The project therefore provides a clear database and application design foundation, while final execution evidence and implementation verification remain important submission tasks.

4.1 Limitations and Future Enhancements

- Complete and execute the final MySQL script, then capture evidence of successful table creation, sample inserts and report outputs.
- Implement and test transaction rollback under simulated failures and concurrency tests for simultaneous seat booking.
- Integrate a payment gateway only after secure credentials, callback validation and reconciliation procedures are designed.
- Add authentication, role-based authorization, audit logs and secure handling of customer contact details.
- Develop a polished GUI or web interface, automated ticket QR codes, email/SMS notifications and online deployment.

- Use EXPLAIN plans and workload-based indexes to optimize commonly used searches and reports.

5. REFERENCES

[1] Oracle, “MySQL 8.4 Reference Manual,” Oracle. [Online]. Available: https://dev.mysql.com/doc/. Accessed: Oct. 15, 2026.

[2] Python Software Foundation, “Python 3 Documentation.” [Online]. Available: https://docs.python.org/3/. Accessed: Oct. 15, 2026.

[3] MySQL, “Connector/Python Developer Guide.” [Online]. Available: https://dev.mysql.com/doc/connector-python/en/. Accessed: Oct. 15, 2026.

[4] E. F. Codd, “A relational model of data for large shared data banks,” Communications of the ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970.

Note: Confirm that each source has been consulted and cited in the final submitted report. The review presentations and the supplied report template were used as project-specific source material.

6. APPENDICES

APPENDIX A — REPRESENTATIVE DATABASE SCRIPT

The following MySQL script is a representative implementation baseline aligned with the proposed logical schema. Review and execute it in the target MySQL environment before submission. The screening-overlap and unique seat-per-screening rules require application-level transactional validation or additional database triggers/locking beyond basic foreign keys.

```
CREATE DATABASE IF NOT EXISTS cinema_booking;
USE cinema_booking;

CREATE TABLE Cinema (
  Cinema_ID INT PRIMARY KEY AUTO_INCREMENT,
  Name VARCHAR(100) NOT NULL,
  Location VARCHAR(150) NOT NULL,
  Total_Screens INT NOT NULL CHECK (Total_Screens > 0)
);

CREATE TABLE Screen (
  Screen_ID INT PRIMARY KEY AUTO_INCREMENT,
  Cinema_ID INT NOT NULL,
  Screen_Name VARCHAR(50) NOT NULL,
  Screen_Type VARCHAR(30) DEFAULT 'Standard',
  Total_Seats INT NOT NULL CHECK (Total_Seats > 0),
  UNIQUE (Cinema_ID, Screen_Name),
  FOREIGN KEY (Cinema_ID) REFERENCES Cinema(Cinema_ID)
);

CREATE TABLE Seat (
  Seat_ID INT PRIMARY KEY AUTO_INCREMENT,
  Screen_ID INT NOT NULL,
  Seat_Number VARCHAR(10) NOT NULL,
  Seat_Type VARCHAR(30) NOT NULL DEFAULT 'Standard',
  UNIQUE (Screen_ID, Seat_Number),
  FOREIGN KEY (Screen_ID) REFERENCES Screen(Screen_ID)
);

CREATE TABLE Movie (
  Movie_ID INT PRIMARY KEY AUTO_INCREMENT,
  Title VARCHAR(150) NOT NULL,
  Genre VARCHAR(50) NOT NULL,
  Duration INT NOT NULL CHECK (Duration > 0),
  Language VARCHAR(50) NOT NULL
);

CREATE TABLE Screening (
  Screening_ID INT PRIMARY KEY AUTO_INCREMENT,
  Movie_ID INT NOT NULL,
  Screen_ID INT NOT NULL,
  Show_Date DATE NOT NULL,
  Start_Time TIME NOT NULL,
  End_Time TIME NOT NULL,
  CHECK (End_Time > Start_Time),
  FOREIGN KEY (Movie_ID) REFERENCES Movie(Movie_ID),
  FOREIGN KEY (Screen_ID) REFERENCES Screen(Screen_ID),
  INDEX idx_screening_date_screen (Show_Date, Screen_ID)
);
```

```
CREATE TABLE Customer (
  Customer_ID INT PRIMARY KEY AUTO_INCREMENT,
  Name VARCHAR(100) NOT NULL,
  Email VARCHAR(150) NOT NULL UNIQUE,
  Phone VARCHAR(20)
);

CREATE TABLE Booking (
  Booking_ID INT PRIMARY KEY AUTO_INCREMENT,
  Customer_ID INT NOT NULL,
  Screening_ID INT NOT NULL,
  Booking_Date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  Status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
  FOREIGN KEY (Screening_ID) REFERENCES Screening(Screening_ID),
  INDEX idx_booking_screening (Screening_ID),
  INDEX idx_booking_customer (Customer_ID)
);

CREATE TABLE Booked_Seat (
  Booked_Seat_ID INT PRIMARY KEY AUTO_INCREMENT,
  Booking_ID INT NOT NULL,
  Seat_ID INT NOT NULL,
  Price DECIMAL(10,2) NOT NULL CHECK (Price >= 0),
  FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID),
  FOREIGN KEY (Seat_ID) REFERENCES Seat(Seat_ID),
  INDEX idx_bookedseat_booking (Booking_ID),
  INDEX idx_bookedseat_seat (Seat_ID)
);

CREATE TABLE Ticket (
  Ticket_ID INT PRIMARY KEY AUTO_INCREMENT,
  Booked_Seat_ID INT NOT NULL UNIQUE,
  Ticket_Code VARCHAR(40) NOT NULL UNIQUE,
  Issue_Date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (Booked_Seat_ID) REFERENCES Booked_Seat(Booked_Seat_ID)
);

CREATE TABLE Concession_Item (
  Item_ID INT PRIMARY KEY AUTO_INCREMENT,
  Item_Name VARCHAR(100) NOT NULL UNIQUE,
  Price DECIMAL(10,2) NOT NULL CHECK (Price >= 0)
);

CREATE TABLE Concession_Order (
  Order_ID INT PRIMARY KEY AUTO_INCREMENT,
  Booking_ID INT NOT NULL,
  Item_ID INT NOT NULL,
  Quantity INT NOT NULL CHECK (Quantity > 0),
  Unit_Price DECIMAL(10,2) NOT NULL CHECK (Unit_Price >= 0),
  Amount DECIMAL(10,2) NOT NULL CHECK (Amount >= 0),
  FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID),
  FOREIGN KEY (Item_ID) REFERENCES Concession_Item(Item_ID)
);

CREATE TABLE Payment (
  Payment_ID INT PRIMARY KEY AUTO_INCREMENT,
  Booking_ID INT NOT NULL,
  Amount DECIMAL(10,2) NOT NULL CHECK (Amount >= 0),
  Payment_Date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  Payment_Method VARCHAR(30) NOT NULL,
  Status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID)
```

```
);

CREATE TABLE Cancellation (
  Cancellation_ID INT PRIMARY KEY AUTO_INCREMENT,
  Booking_ID INT NOT NULL UNIQUE,
  Cancellation_Date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  Reason VARCHAR(255),
  Status VARCHAR(20) NOT NULL DEFAULT 'REQUESTED',
  FOREIGN KEY (Booking_ID) REFERENCES Booking(Booking_ID)
);

CREATE TABLE Refund (
  Refund_ID INT PRIMARY KEY AUTO_INCREMENT,
  Cancellation_ID INT NOT NULL UNIQUE,
  Refund_Amount DECIMAL(10,2) NOT NULL CHECK (Refund_Amount >= 0),
  Refund_Date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  Status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  FOREIGN KEY (Cancellation_ID) REFERENCES Cancellation(Cancellation_ID)
);

INSERT INTO Cinema (Name, Location, Total_Screens)
VALUES ('City Cinema', 'Hyderabad', 2);

INSERT INTO Screen (Cinema_ID, Screen_Name, Screen_Type, Total_Seats)
VALUES (1, 'Screen 1', 'Standard', 100),
       (1, 'Screen 2', 'Premium', 60);

INSERT INTO Seat (Screen_ID, Seat_Number, Seat_Type)
VALUES (1, 'A1', 'Standard'), (1, 'A2', 'Standard'),
       (2, 'P1', 'Premium');

INSERT INTO Movie (Title, Genre, Duration, Language)
VALUES ('The Last Journey', 'Drama', 128, 'Telugu'),
       ('Beyond the Stars', 'Science Fiction', 142, 'English');

INSERT INTO Screening (Movie_ID, Screen_ID, Show_Date, Start_Time, End_Time)
VALUES (1, 1, '2026-10-20', '10:00:00', '12:08:00'),
       (2, 2, '2026-10-20', '13:00:00', '15:22:00');

INSERT INTO Customer (Name, Email, Phone)
VALUES ('Aarav Kumar', 'aarav@example.com', '9000000001'),
       ('Mira Reddy', 'mira@example.com', '9000000002');

INSERT INTO Booking (Customer_ID, Screening_ID, Status)
VALUES (1, 1, 'CONFIRMED');

INSERT INTO Booked_Seat (Booking_ID, Seat_ID, Price)
VALUES (1, 1, 180.00);

INSERT INTO Ticket (Booked_Seat_ID, Ticket_Code)
VALUES (1, 'TKT-2026-0001');

INSERT INTO Concession_Item (Item_Name, Price)
VALUES ('Popcorn', 120.00), ('Water', 40.00);

INSERT INTO Concession_Order (Booking_ID, Item_ID, Quantity, Unit_Price, Amount)
VALUES (1, 1, 1, 120.00, 120.00);

INSERT INTO Payment (Booking_ID, Amount, Payment_Method, Status)
VALUES (1, 300.00, 'UPI', 'SUCCESS');

-- Booking details join
SELECT b.Booking_ID, c.Name AS Customer_Name, m.Title,
       s.Show_Date, s.Start_Time, b.Status
```

```
FROM Booking b
JOIN Customer c ON b.Customer_ID = c.Customer_ID
JOIN Screening s ON b.Screening_ID = s.Screening_ID
JOIN Movie m ON s.Movie_ID = m.Movie_ID;

-- Occupancy by screening (based on allocated seats)
SELECT s.Screening_ID, m.Title, sc.Screen_Name,
       COUNT(bs.Booked_Seat_ID) AS Seats_Booked,
       sc.Total_Seats,
       ROUND(100.0 * COUNT(bs.Booked_Seat_ID) / sc.Total_Seats, 2) AS
Occupancy_Percent
FROM Screening s
JOIN Movie m ON s.Movie_ID = m.Movie_ID
JOIN Screen sc ON s.Screen_ID = sc.Screen_ID
LEFT JOIN Booking b ON b.Screening_ID = s.Screening_ID
LEFT JOIN Booked_Seat bs ON bs.Booking_ID = b.Booking_ID
GROUP BY s.Screening_ID, m.Title, sc.Screen_Name, sc.Total_Seats;

-- Movie-wise ticket revenue
SELECT m.Title, COUNT(t.Ticket_ID) AS Tickets_Issued,
       COALESCE(SUM(bs.Price), 0) AS Ticket_Revenue
FROM Movie m
JOIN Screening s ON s.Movie_ID = m.Movie_ID
LEFT JOIN Booking b ON b.Screening_ID = s.Screening_ID
LEFT JOIN Booked_Seat bs ON bs.Booking_ID = b.Booking_ID
LEFT JOIN Ticket t ON t.Booked_Seat_ID = bs.Booked_Seat_ID
GROUP BY m.Movie_ID, m.Title;

-- Customers with more than three bookings
SELECT c.Customer_ID, c.Name, COUNT(b.Booking_ID) AS Booking_Count
FROM Customer c
JOIN Booking b ON b.Customer_ID = c.Customer_ID
GROUP BY c.Customer_ID, c.Name
HAVING COUNT(b.Booking_ID) > 3;

-- Concession revenue by item
SELECT ci.Item_Name, SUM(co.Quantity) AS Units_Sold,
       SUM(co.Amount) AS Revenue
FROM Concession_Item ci
JOIN Concession_Order co ON co.Item_ID = ci.Item_ID
GROUP BY ci.Item_ID, ci.Item_Name;

CREATE OR REPLACE VIEW movie_sales AS
SELECT m.Movie_ID, m.Title, COUNT(t.Ticket_ID) AS Tickets_Issued,
       COALESCE(SUM(bs.Price), 0) AS Ticket_Revenue
FROM Movie m
JOIN Screening s ON s.Movie_ID = m.Movie_ID
LEFT JOIN Booking b ON b.Screening_ID = s.Screening_ID
LEFT JOIN Booked_Seat bs ON bs.Booking_ID = b.Booking_ID
LEFT JOIN Ticket t ON t.Booked_Seat_ID = bs.Booked_Seat_ID
GROUP BY m.Movie_ID, m.Title;
```

APPENDIX B — APPLICATION CONNECTIVITY EXAMPLE

```
import os
import mysql.connector

db_cfg = {
    "host": os.environ["DB_HOST"],
    "user": os.environ["DB_USER"],
    "password": os.environ["DB_PASSWORD"],
    "database": os.environ["DB_NAME"],
}
```

```
conn = mysql.connector.connect(**db_cfg)
try:
    cur = conn.cursor(dictionary=True)
    cur.execute(
        "SELECT * FROM Screening WHERE Movie_ID = %s",
        (movie_id,)
    )
    rows = cur.fetchall()
    # Process rows in the application interface.
finally:
    cur.close()
    conn.close()
```

APPENDIX C — SOURCE CODE, SCREENSHOTS AND ADDITIONAL EVIDENCE

Application repository: Not supplied in the provided review materials. Insert the actual repository URL or source folder name before submission.

Screenshots to attach: (1) cinema/movie data entry, (2) screening search and seat availability, (3) booking and ticket issue, (4) concession/payment or cancellation/refund, and (5) one report output. Use actual screenshots from the running application and label each figure.

Additional verification checklist:

- Execute Appendix A in the target MySQL version and correct any environment-specific issues.
- Confirm that the unique seat-per-screening rule is enforced correctly when two requests occur concurrently.
- Test that overlapping screening intervals are rejected, including shows that cross midnight if such shows are supported.
- Test transaction rollback after a deliberate mid-booking failure.
- Update all test statuses with observed outcomes and attach evidence.
- Verify the student roll number and add faculty name, repository link and any required institutional details.